

Capítulo 4: Descubrimiento

Discovery

Understanding Distributed Systems — Roberto Vitillo

Contexto: ¿dónde estamos?

En los capítulos anteriores usted aprendió a:

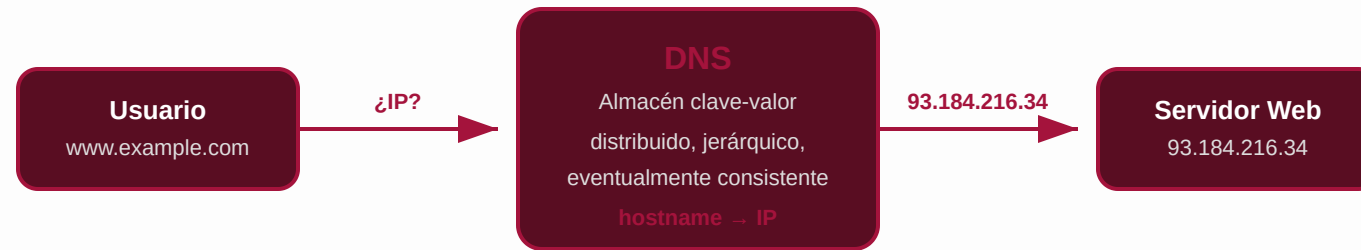
- **Cap. 2 — Reliable Links**: crear un canal confiable con TCP
- **Cap. 3 — Secure Links**: proteger el canal con TLS

Pero hay un problema previo que no hemos resuelto:

*Para abrir una conexión TCP con un servidor, usted necesita conocer su **dirección IP**. ¿Cómo la obtiene?*

La respuesta es el **DNS** (Domain Name System) — el sistema de descubrimiento de Internet.

DNS: la guía telefónica de Internet



DNS: La guía telefónica de Internet

Traduce nombres legibles por humanos a direcciones IP

¿Por qué no usar simplemente direcciones IP?

Las direcciones IP son difíciles de memorizar para los humanos, pero hay razones más importantes:

- **Las IPs cambian**: un servidor puede moverse de máquina, de proveedor de hosting, o de región geográfica. El nombre de dominio permanece igual.
- **Balanceo de carga**: un mismo nombre puede resolver a **múltiples IPs** diferentes, distribuyendo el tráfico.
- **Redundancia**: si un servidor falla, se actualiza el registro DNS para apuntar a otro.
- **Abstracción**: desacopla la identidad del servicio (nombre) de su ubicación (IP).

*DNS es un nivel de **indirección** fundamental en la arquitectura de Internet.*

La resolución DNS

¿Qué pasa cuando usted escribe una URL en el navegador?

Paso 1: la caché del navegador

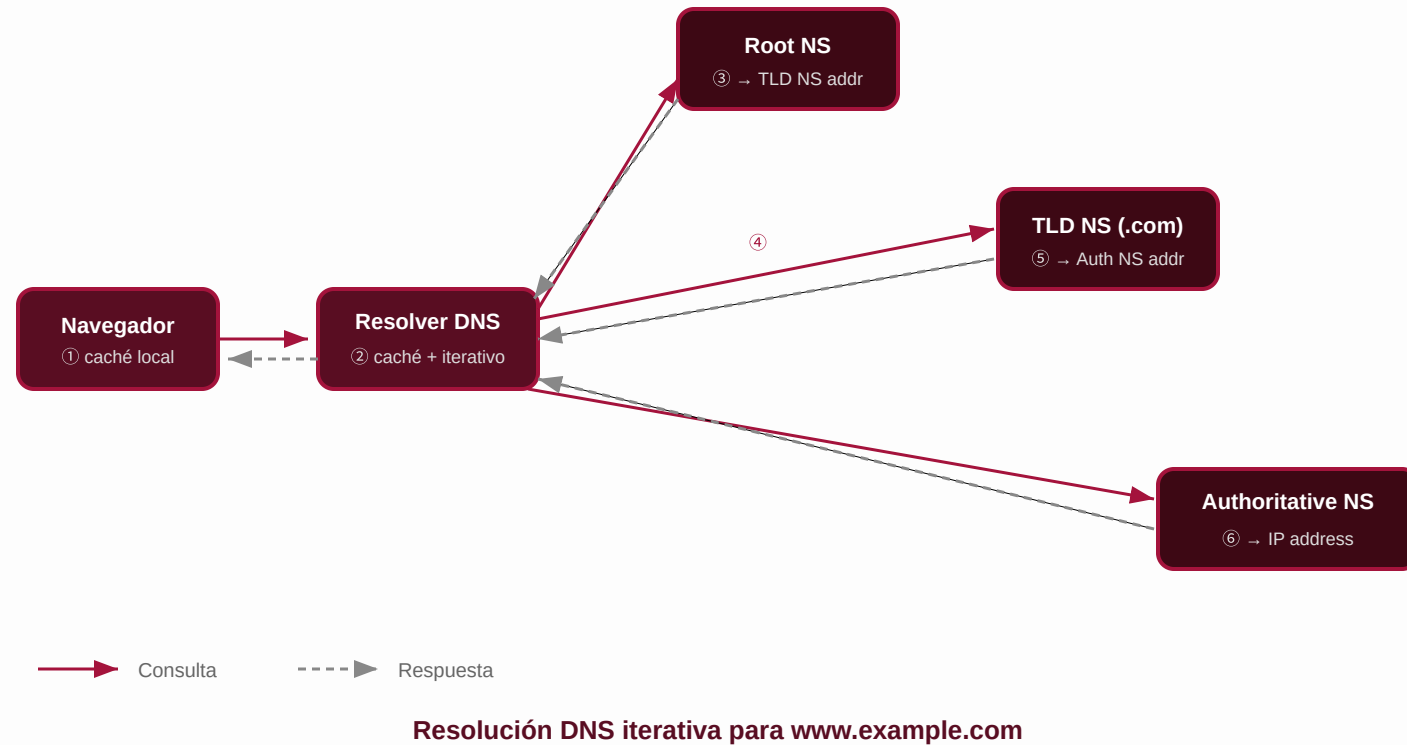
Cuando usted escribe `www.example.com` en su navegador:

1. **El navegador revisa su caché local** — si ya resolvió este nombre recientemente, tiene la IP guardada y la reutiliza.
2. Si no la tiene, consulta la **caché del sistema operativo** (que puede tener un archivo `hosts` local con entradas estáticas).
3. Si tampoco, envía la consulta a un **resolver DNS** — típicamente un servidor DNS de su proveedor de Internet (ISP) o un resolver público como `8.8.8.8` (Google) o `1.1.1.1` (Cloudflare).

La mayoría de las consultas DNS se resuelven desde alguna caché sin llegar a los servidores autoritativos.

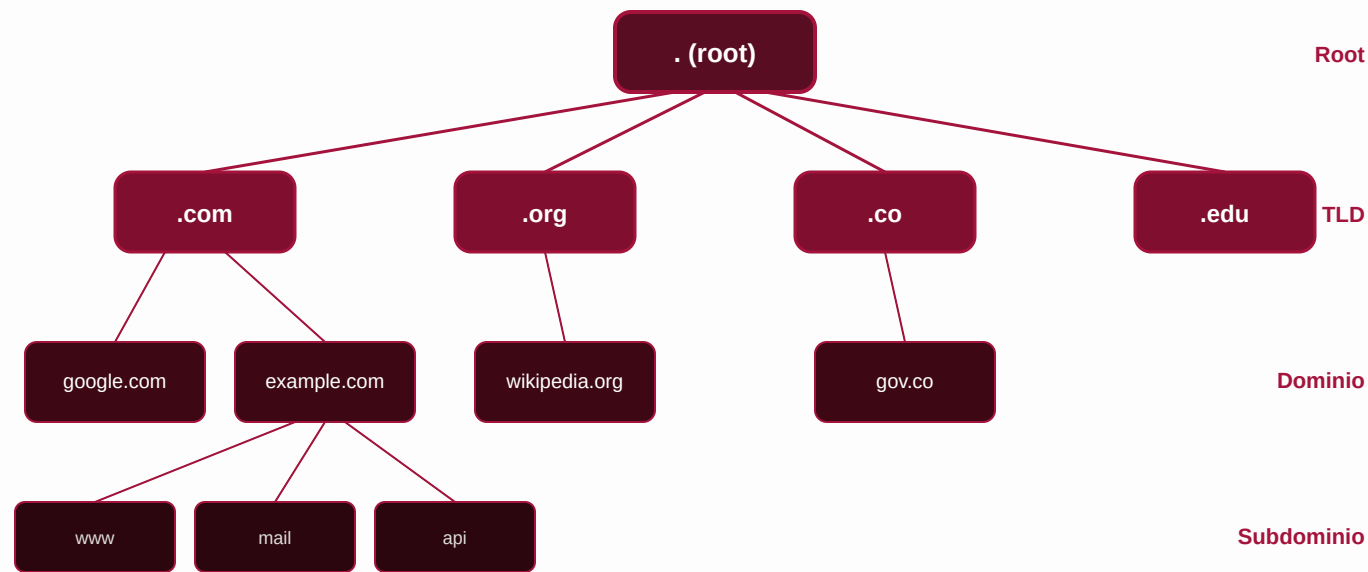
Paso 2: resolución iterativa

Si el resolver no tiene la respuesta en su caché, inicia una **resolución iterativa**:



La jerarquía DNS

DNS está organizado como un **árbol invertido**:



Cada nivel del árbol es gestionado por servidores diferentes, lo que permite **delegar la responsabilidad** y **escalar** a miles de millones de nombres.

Los 13 servidores raíz

Los **root name servers** son el punto de entrada cuando el resolver no tiene información cacheada:

Letra	Operador	Instancias globales
A	Verisign	10+
B	USC-ISI	1
F	ISC	200+
J	Verisign	200+
K	RIPE NCC	70+
L	ICANN	170+
M	WIDE Project	10+

En total hay **más de 1,500 instancias** distribuidas por todo el mundo usando **anycast**: la misma IP enruta al servidor más cercano geográficamente.

Tipos de registros DNS

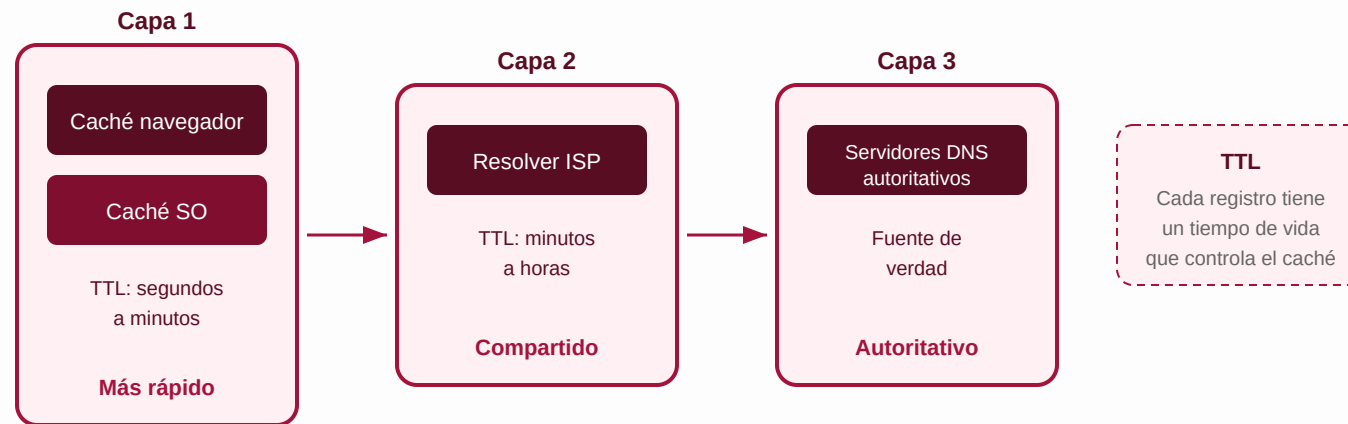
No solo se resuelven nombres a IPs. DNS almacena varios tipos de registros:

Tipo	Significado	Ejemplo
A	Hostname → IPv4	www.example.com → 93.184.216.34
AAAA	Hostname → IPv6	www.example.com → 2606:2800:...
CNAME	Alias → otro hostname	blog.example.com → ex.cdn.net
MX	Dominio → servidor de correo	example.com → mail.example.com
SRV	Servicio → host + puerto	_http._tcp.ex.com → web1:8080

Caché y consistencia eventual

El rol del TTL y sus implicaciones

Las capas de caché DNS



Eventualmente consistente: los cambios no se propagan instantáneamente

Cada respuesta DNS incluye un **TTL** (Time to Live) que indica cuánto tiempo puede ser cacheada.

Consistencia eventual en DNS

El DNS es **eventualmente consistente**: si usted cambia la IP asociada a un nombre de dominio, el cambio **no se refleja inmediatamente** en todos los clientes.

¿Por qué?

- Cada caché retiene la respuesta anterior hasta que expire su TTL.
- Un TTL de 1 hora significa que pueden pasar hasta 60 minutos antes de que todos los clientes vean el nuevo valor.
- Algunos resolvers ignoran el TTL y cachean por más tiempo.

Implicaciones prácticas:

- Al migrar un servicio, **reduzca el TTL antes** de la migración (e.g., bájelo a 60 segundos días antes).

- Después de la migración, espere al menos un ciclo de TTL antes de apagar el servidor

TTL: el compromiso rendimiento vs. frescura

TTL corto (60s)	TTL largo (1h+)
Los cambios se propagan rápido	Los cambios tardan en propagarse
Mayor carga en servidores DNS	Menor carga en servidores DNS
Mayor latencia percibida (más consultas)	Mejor rendimiento (más cache hits)
Ideal para migración o failover	Ideal para servicios estables

*El TTL es un ejemplo clásico del compromiso entre **frescura de datos** y **rendimiento**.*

DNS como sistema distribuido

DNS es uno de los sistemas distribuidos **más exitosos** de la historia. Veamos sus propiedades:

Propiedad	Cómo la implementa DNS
Distribuido	Miles de servidores en todo el mundo
Jerárquico	Root → TLD → Autoritative → Subdominio
Escalable	Cada nivel delega al siguiente; anycast
Eventualmente consistente	Caché con TTL en cada nivel
Tolerante a fallos	Múltiples servidores por nivel; anycast
Descentralizado	Cada organización administra su zona

DNS es un caso de estudio perfecto de los principios de diseño de sistemas distribuidos que veremos a lo largo del curso.

Más allá de la resolución básica

Aspectos avanzados del descubrimiento de servicios

DNS y la seguridad

DNS fue diseñado en los años 80 sin consideraciones de seguridad. Esto ha generado problemas:

- **DNS Spoofing / Cache Poisoning**: un atacante inyecta registros falsos en la caché de un resolver, redirigiendo tráfico a un servidor malicioso.
- **DNS Hijacking**: un atacante modifica los registros DNS directamente en el proveedor.
- **DNS Amplification DDoS**: se usa DNS como amplificador de tráfico en ataques de denegación de servicio.

Soluciones:

- **DNSSEC** (DNS Security Extensions): firma criptográficamente los registros DNS para verificar su autenticidad.
- **DoH** (DNS over HTTPS) / **DoT** (DNS over TLS): cifra las consultas DNS para prevenir espionaje y manipulación.

Descubrimiento de servicios en la práctica

En sistemas distribuidos modernos (microservicios, cloud), el descubrimiento de servicios va **más allá de DNS**:

Mecanismo	Ejemplo	Uso típico
DNS clásico	Registros A/CNAME	Servicios públicos (web)
DNS interno	CoreDNS, Route53	Servicios dentro de un cluster
Service Registry	Consul, Eureka, etcd	Microservicios dinámicos
Service Mesh	Istio, Linkerd	Descubrimiento + balanceo + seguridad

*En entornos cloud, los servidores se crean y destruyen constantemente. El descubrimiento de servicios debe ser **dinámico y automático**.*

Ejemplo práctico: `nslookup` y `dig`

Usted puede explorar DNS desde la terminal:

```
# Consulta básica
nslookup www.google.com

# Consulta detallada con dig (Linux/Mac)
dig www.google.com

# Ver registros MX (correo)
dig example.com MX

# Ver toda la cadena de resolución
dig +trace www.example.com
```

Ejercicio sugerido: use `dig +trace www.example.com` y observe cómo la consulta viaja desde el root NS hasta el servidor autoritativo, nivel por nivel.

Puntos clave del capítulo

1. **DNS** es un almacén clave-valor distribuido, jerárquico y eventualmente consistente que traduce nombres de dominio a direcciones IP.
2. La **resolución iterativa** recorre la jerarquía: Root → TLD → Autoritativo, con caching en cada nivel.
3. El **TTL** controla cuánto tiempo se cachea una respuesta, estableciendo un compromiso entre rendimiento y frescura.
4. DNS es **eventualmente consistente**: los cambios no son instantáneos. Esto tiene implicaciones importantes para migraciones y failover.

5. DNS fue diseñado sin seguridad; **DNSSEC**, **DoH** y **DoT** abordan estas limitaciones.
6. En sistemas distribuidos modernos, el descubrimiento de servicios extiende DNS con registros dinámicos, health checks y service meshes.

Para profundizar

- **RFC 1034 / 1035** — Especificación original de DNS
- **RFC 4033-4035** — DNSSEC
- Herramienta: `dig +trace` para ver la resolución completa
- Herramienta: `nslookup` disponible en todos los sistemas operativos
- Cloudflare Learning Center: <https://www.cloudflare.com/learning/dns/>

Ejercicio sugerido: investigue qué registros DNS tiene el dominio de su universidad. Use `dig` o `nslookup` para encontrar registros A, MX, TXT y NS. ¿Cuántos servidores de correo tiene?

¿Preguntas?

Capítulo 4: Descubrimiento — *Discovery*